

LGBT Center Library Catalog

Boris Brondz, Daniel Burwell, Hannah Hejazi, Deborah Park, Natalie Qiu, Fei Triolo

Problem Statement / Specifications / Requirements

The CWRU LGBT Center hosts a small library containing 4 shelves of books, DVDs and other borrowable documents. The current Library Management System is a Google Sheet containing most of the existing catalog, and a Google Form for patrons to check out and return books. This current system has significant overhead for both patrons who struggle to find and checkout relevant materials as well as LGBT Center employees who are required to meticulously update the spreadsheet and personally track patrons' due dates.

Solutions

We intend to develop a full-stack web application and database that provides a searchable interface for patrons to browse relevant material, checkout, renew, and track their due dates. The application should also contain accessible Library Management features for LGBT Center Staff to update, add and remove items without having technical knowledge of the backend. We hope to utilize university resources to host the application on dedicated server space with a public-facing website for future LGBT Center staff and visitors to use.

The application interface should be easy for the average reasonable person to use, secure to protect the privacy of patrons, and appealing to look at. The database structure should be robust enough to need minimal maintenance in the future, to minimize the burden on the LGBT Center staff. We will nonetheless provide documentation for future updates.

Background & Related Work

Fei has worked on similar Library Management backend projects for courses such as CSDS293—Java-based Library Management software architecture, and CSDS341—MySQL database and rudimentary JDBC front end.

We are drawing inspiration from existing public library catalogs such as the Heights Libraries (<https://heightslibrary.org>), the Cuyahoga County Public Library (<https://cuyahogalibrary.org/>), the King County Library System (<https://kcls.org/>), and the Cleveland Public Library (<https://cpl.org>).

For the backend and database creation, we are using Go and MySQL, and React.js for the front-end interface. Additionally, we may experiment with a Spring Boot back-end to prioritize security.

Finally, we will be utilizing the university's CAS Single Sign-On system in order to manage logging into and out of the library catalog, documentation for which can be found at

<https://case.edu/utech/help/knowledge-base/cwru-network-id-passphrase/cas-single-sign-cwru-kba>.

Preexisting database schema:

As stated above, the current method used to keep track of the library catalog is a spreadsheet which can be browsed by potential clients and must be manually updated by staff members, which can be found at

https://docs.google.com/spreadsheets/d/1tbhRvgI_Dn5JIICZVFxTj26xtTF0JUEa-iRT5JDvmvs/htmlview. Patrons can check out books by filling out the Google form linked in the spreadsheet, at <https://forms.gle/pk3VKTN5Bux59rTp9>.

Completed Work

1. Contacted UTech to negotiate for server space
2. Set up a demonstrative system with barebones:
 - Database with sample data
 - Catalog
 - ISBN number
 - searchable info (author, title, date published, format content tags)
 - Total copies + copies available
 - Users
 - caseID
 - Name (for display)
 - Level of auth (use eNUM?)
 - Checkouts (linking table) – delete entry when book is checked back in, to preserve privacy.
 - caseID
 - ISBN
 - Date checked out
 - Javascript web interface with
 - **MUI table (supports CRUD operations)** to show users and library items
 - Account function: 3 levels of authentication

- Manager auth: **update** user authorizations, **update** checkout restrictions (max checkouts, duration of checkouts, max number of renewals)
- Employee auth: **update** check-outs, **update/create/remove** catalog items
- Patron auth: **read** catalog entries, **write** checkouts

2.1. Developing the first iteration of the REST API (backend server - Golang).

- All basic **REST** endpoints (/books, /books/, /users, /users/, /authors, /loans, /loans/, /search, /tags) - all compatible with **GET, PUT, POST, DELETE, PATCH** operations.
- Write the proper documentation for the frontend team so they can understand how they can set up the backend server locally and incorporate **REST API** calls on the frontend (via **axios/fetch libraries + cURL**).
- Develop a “gateway” system to easily deploy different backend services that will be further utilized in the future, with **special runtime flags/parameters** for ease of use in the terminal.
- Add **CORS** support so the backend and frontend can be deployed and run on different ports for local testing.
- Develop an offset-based pagination system to minimize overhead for any **GET** requests to minimize bandwidth (with a set default limit of 10 **JSON** objects per page).
- Create a simple **rate-limiting system** that by default allows for at max 10 requests per second in the meantime.
- Add a simple **Makefile** for members with **Linux/Unix** based systems to simplify runtime commands.

2.2. Creating a database schema

Work to be done (Timeline)

3. Outline poster
4. Update poster, update documentation
5. Iteratively improve this system, work with UTech
 - Integrate frontend with backend/database server
 - Rectify frontend with backend/database to deliver full user features
 - Verify login w/ CASE SSO
 - Webservices:
 - Email confirmation upon checking out a book
 - Email notification for overdue books
 - Submission form for suggesting new books, which emails to the Managers and Employees

- SANITIZING Inputs to prevent SQL injection
6. Make it look good
- Use design kits (Tailwind CSS/Bootstrap/etc..) with a front-end specific framework (React/Angular/Vue)

Throughout work, we will update the poster.

Management Plan & Accountability

This project can be divided into 2 teams: database backend and web interface front end. The role of the frontend team will be to create the look and feel of the website using HTML, CSS, and JavaScript. The backend team will be responsible for managing the creation of the database, as well as managing database queries sent from the frontend (including sanitizing inputs as necessary). Additionally, they will have to handle users and authentication through the CAS system. Someone in the group will also need to contact UTech to discuss hosting the site on the school's servers.

In preparation for the demo on October 17, we should have a simple version of the database setup with sample entries, as well as a draft of the website design, and a way for the website to communicate with the database to simulate checking out, renewing, and checking in books. Sample users will be utilized without needing to be authenticated with the CAS system for the purpose of the demo.

In between the demo and the end of the semester, we will integrate the CAS system into the account system and clean up the look and feel of the website. We will also have the website hosted on a server by that point so that it can be used by anyone wishing to check books out of the LGBT Center library.

In order to keep on track with this plan, we will keep in regular contact over a group chat and aim to have at least one in-person meeting per week, where we can evaluate the plan moving forward and address any issues which have arisen since the last meeting. We will also keep track of who accomplished what with a spreadsheet:

 [Team Planning Spreadsheet](#) .

Accountability

As of the first check-in, we have created our prototype. Fei worked on the database schema and set up prepared statements, and filmed the demonstration video. Boris and Dan worked on the Go backend. Boris created the framework for the API server and worked on pagination and the API endpoints for handling books (including individual books by using the book's ID), users, and searching. Dan built the API endpoints for handling loans, including renewing and returning (i.e. individual loans by ID), and authors. Natalie and Deborah worked on the react.js front end, with Natalie implementing the tables and Deborah implementing the example login system. Hannah

was assigned to work on the project report and researching integration between the database and front-end application, but has otherwise not contributed much. Natalie also spoke with UTech and is in the middle of obtaining server space for the project.

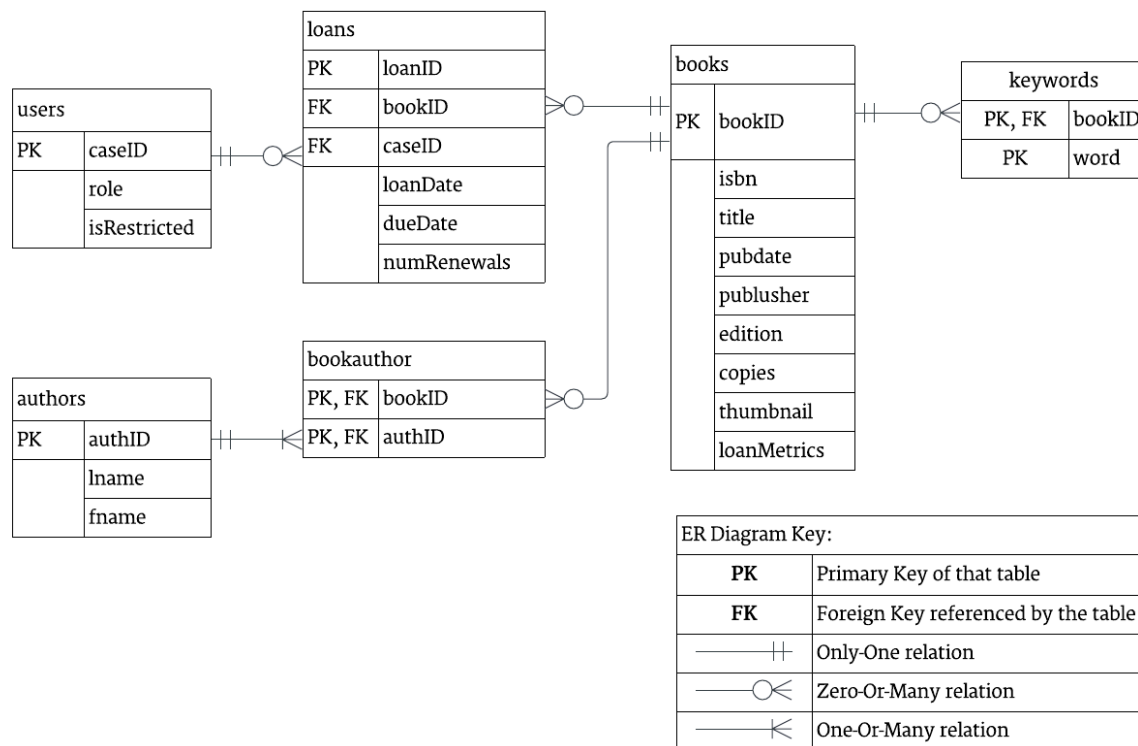
Relational Database Schema Development

Based on previous project experience, the database schema was carefully designed to maximize a number of considerations including space efficiency, backend integration, and design requirements.

Our catalog must be easily searchable, both by intrinsic properties of a book (i.e. title, publisher information, authors), and by metadata such as genre, subject matter, medium, etc. We decided to abstract this metadata into a keyword system in which each book can be related to multiple user-defined keywords. This allows flexibility for the organization to determine what relevant metadata is important for them to associate with each book. Additionally we decided to store author data separately from individual books, related to each other by the bookauthor transition table. This enables a many-to-many relation between books and authors which minimally accounts for cases where a book has multiple authors, each equally searchable, or where the same author contributes to multiple books in the collection.

Our primary concern for the user validation and loan tracking is user privacy. Due to social stigma associated with the LGBT community, patrons may have aversions to the possibility of their patronage of center resources being traced back to their personal identities. Additionally, many patrons may otherwise be uncomfortable seeing the same name associated with their University account used in the context of LGBT community spaces (such as a transgender student who is privately out amongst regular LGBT Center attendants but still wishes to remain closetted in their larger academic life). Ultimately, we decided to store as little user information as possible without sacrificing critical functional requirements (i.e. caseID, user role, and loan restriction status). This rationale also informed our decision to make loan information temporal—that is when a loan is returned, we discard all receipts of that loan associated with that user.

Below is the full Entity Relation (ER) Diagram for our database schema:



While the schema itself was relatively straightforward to design and implement in a MySQL instance, we faced some technical challenges in the development workflow. While we were waiting to get server space from the university, we had nowhere to host our development instance. Since github is not optimal for hosting relational databases, the group members who focused on database development opted to locally host their own instances of the database in their machines. This presented difficulties in version control, having to manually update the schema across all active instances whenever new database definition queries were written. Additional difficulties proved in integration between the database and backend.

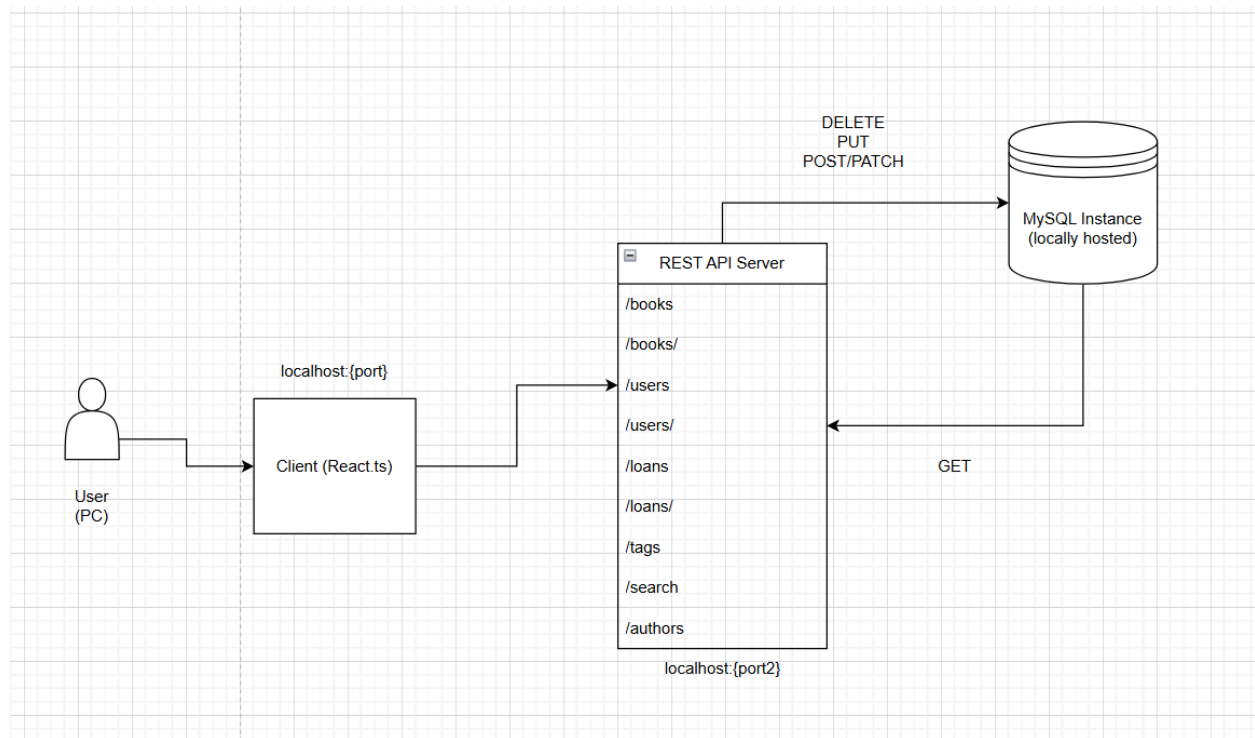
Since the schema is pretty much complete, future responsibilities should be to work with UTech on hosting. Dedicated server space is critical for the database end of this project to function as desired. Additionally, as we make progress integrating the front and backend it is possible that bugs will arise that are exacerbated by the database schema and fixing those bugs requires close collaboration between teams. Ultimately, we can now afford to focus work away from the database and towards other parts of the project

Backend API Development

To create a layer of abstraction between the frontend and database, we devised a Golang-based backend server to facilitate all communication between the frontend and database. Aside from being a standard procedure when developing web applications, the creation of a REST API allows for:

1. Secure transactions (prepared SQL queries that are parametrized – no need to worry about basic attacks such as SQL injection).
2. Limit concurrent access to the application to prevent denial of service attacks (through the introduction of rate limiting)
3. Effectively tackle the issue of scalability. The backend can be scaled independently of the database and frontend to accommodate traffic relative to other components.
4. Utilize the API as a standalone application that can be utilized as a service for anyone else interested in / intending to work with the LGBT center on anything that utilizes data in the LGBT Library Catalog.

System Design Diagram for the current iteration of the LGBT Library Catalog App:



The reasoning behind using Golang as opposed to a language like Java or JavaScript or even Python relates to the concept and purpose of Go as a programming language. Go is much more lightweight and versatile, and supports cross-compilation with different

CPU architectures (x86, x64, ARM, etc..). The ease of use of Go for backend development allows minimal overhead in terms of packages and libraries as well.

Role Members Assigned: Boris Brondz, Daniel Burwell

Boris Responsibilities:

1. Build API endpoints for books, search, and users
2. Incorporate pagination into handlers
3. Write documentation for the backend server for the frontend team

Daniel Responsibilities:

1. Build API endpoints for loans and authors
2. Handle code for checking out, renewing, and returning books

NOTE: All documentation for the backend API is located in our GitHub repository in the backend folder. The documentation is a README file containing instructions to set up and run the backend server on a local machine. The README also contains sample API endpoints to help the frontend team understand how they can implement calls to the backend via fetch, axios libraries (examples for both provided) + cURL for simple testing.

In terms of core functionality, the REST API server is mostly complete. The main priorities after the progress report due date (10/17) are laid out in the list of future plans below.

Future Plans:

1. Implement CWRU-based CAS (Central Authentication Service) authentication as a middleware layer between the client and the (REST API) server to handle CWRU CaseID user authentication and login so it can later eventually be used by the campus community.
2. Work on a comprehensive logging system (possibly a centralized logging system) to handle and manage messages/errors produced by REST API calls on the server.
3. Comprehensive unit/integration testing suite that can be run during the “build” step for application deployment to a dedicated UTech server.
4. Re-modularize backend code to reduce boilerplate and better follow Golang syntax conventions.

5. Split the monolithic back-end into microservices, since each endpoint depends on their own table(s).

Fullstack Frontend Design

Our web-based frontend uses Vite and React. Our front end prototype prioritized simplicity for two reasons. First, we are aware that members of the LGBTQ center would enjoy not wading through clunky UI. Secondly, we were short on time. That being said, we will add an actual style to the page to truly give it a personality, or we will die trying.

Currently, the actual contents of the page are built with material-ui's react-table component, which supports CRUD operations and pagination. This code was modified from a preexisting project of Natalie's. The login modal is currently a simple react component to demonstrate functionality, though the final version will be similarly simple— redirecting the user to the CAS Single Sign-On and letting the backend handle loading user data.

We need to test whether or not the single scroll with paginated tables is too simple, and if we should separate different tables into more tabs. The matter of properly implementing our backend should be as simple as updating the helper functions to make API calls.

Prioritized, our to-do list becomes:

1. Properly call from API
2. Paginate /route the tables instead of having 1 scroll
3. Implement assets to properly convey the LGBTQ center's personality

Considering that this is a much shorter list than the backend, and that much of it depends on the backend functioning properly, we are considering moving one or both of us to work on the backend.